ITS SURABAYA — GRAPH THEORY CODING TASK

Task 3: Knight's Tour Simulator

Hamiltonian Path Search with DFS, Backtracking and Warnsdorff's Rule

1. Problem Definition & Graph Representation

The objective is to generate a sequence of legal knight moves on a user-defined rectangular chessboard such that **every square is visited exactly once**. The user specifies the board dimensions and the knight's starting coordinates. If no complete tour exists, the program prints `path not found`.

We model the board as an **undirected, unweighted graph** $G = (V, E)$. Every square (x, y) is a vertex, where $0 \le x < columns$ and $0 \le y < rows$. An edge connects two squares if the knight can move between them in one legal move: an offset of $(\pm1, \pm2)$ or $(\pm2, \pm1)$. Moves outside the board are excluded. Because knight moves are reversible, no edge direction is needed.

The program stores the graph as an **adjacency list**, implemented by a Python dictionary mapping each coordinate to its valid neighbors. The solution corresponds to a **Hamiltonian path** in this graph: a path containing all vertices without repetition. The assignment does not require returning to the starting square, so a Hamiltonian cycle is not necessary.

2. Algorithm Explanation & Implementation

A. Graph Construction and Legal Knight Moves

The function `cavalier(x, y, columns, rows)` applies the eight possible knight offsets and keeps only coordinates satisfying the board boundaries. The main solver builds this neighbor list for every square once, using a dictionary comprehension. Each vertex has at most eight adjacent vertices.

B. Preliminary Feasibility Checks

Before exploring candidate tours, the implementation rejects some impossible cases using **necessary (but not sufficient)** conditions:

- **Isolation:** on a board with more than one square, no vertex may have degree zero.
- **Degree-one vertices:** there may be at most two, and if there are exactly two, the selected start must be one of them.
- **Color parity:** every knight move changes square color; if the board has an odd number of squares, the start must lie on the more numerous color.
- **Connectivity:** a DFS reachability check ensures all board squares can be reached from the starting vertex.

C. Depth-First Search with Backtracking

The solver begins with `path = [start]` and `visited = {start}`. At each recursive call, it considers only unvisited adjacent squares. It appends a candidate and marks it visited, then continues recursively. If the path reaches $columns \times rows$ squares, a complete tour has been found. If a branch becomes blocked, the solver removes the last vertex from both structures and tries another neighbor. Exhausting every branch means no Hamiltonian path exists from the requested start.

D. Warnsdorff Move Ordering

To guide the search, candidate squares are sorted by their number of **currently unvisited onward neighbors**. The algorithm tries the candidate with the fewest onward moves first, reducing the chance of leaving difficult squares until the end. This is **Warnsdorff's heuristic**; it changes the order of DFS exploration but does not replace backtracking or guarantee polynomial running time.

ITS SURABAYA — GRAPH THEORY CODING TASK

Task 3: Evaluation & Complexity

Implementation behavior, sample output and performance analysis

3. Execution, Output and Validation

The implementation is written in Python, using only the standard library. Its entry point is `knights_tour.py`. The input consists of two lines: `columns rows`, followed by zero-based `start_x start_y`. For a solution, it prints one `x y` coordinate per line, in visit order. Otherwise it prints `path not found`.

Example: 5 × 5 board starting at (2, 2)

25	10	15	4	19
16	5	18	9	14
11	24	1	20	3
6	17	22	13	8
23	12	7	2	21

Input

```
5 5
2 2
```

First five output coordinates

(2, 2) → (3, 4) → (4, 2)
→ (3, 0) → (1, 1)

The numbered board shows the visit order. Square **1** is the starting location. All 25 squares are visited once, using 24 legal moves.

For example, input `4 4` and `0 0` produces `path not found`. The implementation also handles a 1 × 1 board as a single-square tour.

4. Computational Complexity Analysis

Let **N = columns × rows** be the number of squares. Since each square has at most eight neighbors, the graph has at most $8N/2 = 4N$ undirected edges.

Algorithmic component	Time complexity	Space complexity	Explanation
Graph construction	O(N)	O(N)	Generate at most 8 neighbors per square.
Feasibility checks	O(N)	O(N)	Degree, parity and connectivity checks.
DFS + Warnsdorff	$O(8^N)$ worst case	O(N)	Potential exponential backtracking; only constant-size neighbor sorts.
Overall solver	$O(8^N)$ worst case	O(N)	Exponential search dominates; graph and recursion stack scale linearly.

The exponential upper bound is intentionally conservative. Warnsdorff's rule often finds a valid tour quickly, but difficult or impossible cases may still require extensive search. Python's recursion depth can limit very large boards.

5. Test Cases and Reproducibility

The repository includes `test_knights_tour.py` with seven passing unit tests, covering the assignment example (5 × 5), a standard 8 × 8 board, a rectangular 3 × 4 board, a 4 × 4 no-solution case, an invalid starting color for a 5 × 5 board, a 1 × 1 board, and an out-of-bounds start. Tests validate the number and uniqueness of visited squares, the starting position, board boundaries and the legality of consecutive moves.

Run: `python knights_tour.py | ` **Tests:** `python -m unittest test_knights_tour.py`

AI-use disclosure: AI assistance was used in drafting and reviewing this Task 3 solution and report. The relevant prompt history should be attached as required by the assignment.